FOODRESCUE TN

SAVE GOOD FOOD.
SHARE MORE.
WASTE LESS.

TECHNICAL SPECIFICATION / CAHIER DES CHARGES

AWD • Validation 1 • Proposed design

FROM SURPLUS TO SOCIAL IMPACT

Donors → Charities → Volunteer pickups → Measurable food saved

Project identification

Field	Information
Module / school	AWD / ESPRIT School of Engineering
Academic year	2026–2027
Class / group	[CLASS] / [GROUP]
Team members	[MEMBER 1], [MEMBER 2], [MEMBER 3], [MEMBER 4], [MEMBER 5], [MEMBER 6]
Instructor	[INSTRUCTOR NAME]
Repository	[PUBLIC GITHUB LINK]
Status	Design proposal; implementation evidence not yet confirmed

Document version: 2.0 • 10 November 2026

01 Overview & functional scope

Short, realistic scope for the Session 4 design review.

Table of contents

01 Overview & functional scope | 02 Service breakdown | 03 Architecture | 04 UML design (4 pages) | 05 Communication & sequences | 06 API contracts

Why this project?

Restaurants, bakeries, hotels and stores in Tunisia discard safe surplus food while associations need supplies. FoodRescue TN lets verified donors publish offers, matches them with compatible charities and volunteer drivers, and also sells discounted surprise baskets to citizens.

Users and core requirements

Actor	Feature	Simple requirement
Donor	Publish surplus	Create and update offers with quantity, expiry and pickup window
Association	Receive donations	Specify needs, accept matches and monitor collection
Volunteer driver	Transport food	Set availability; collect and deliver assigned pickups
Citizen	Buy rescue basket	Browse, reserve and collect discounted baskets
Administrator	Build trust	Verify partners, monitor safety and moderate accounts
All roles	Identity / alerts	Log in securely and receive event notifications
Admin / partners	Impact dashboard	View kilograms saved, meals estimated and CO2e avoided

Three priority scenarios

1. Charity rescue: bakery posts 20 kg → verified charity is matched → volunteer collects → Impact Analytics updates totals.
2. Citizen basket: restaurant offers a discounted basket → customer reserves available stock → pickup code is redeemed.
3. Trust review: new donor registers → admin checks documents and food safety → verified donor can publish offers.

02 Microservice breakdown

One bounded context and one separately owned data store per service.

Service	Owned entities / responsibilities	Persistence and reason
1 · Donation Offer	DonationOffer, FoodItem, StockReservation, PickupLocation, OfferPhoto; quantity is authoritative	MongoDB · flexible offers and items
2 · Matching & Pickup	AssociationNeed, DonationMatch, Pickup, DriverAssignment, PickupStatusEvent	PostgreSQL + PostGIS · transactions / distance
3 · Partner Registry & Trust	Partner, PartnerBranch, VerificationCase, FoodSafetyCheck, PartnerReview	PostgreSQL · approval and audit records
4 · Impact Analytics	ImpactEvent, ConversionFactor, ImpactSummary, DemandForecast, ReportExport	PostgreSQL + Redis cache · aggregates
5 · Volunteer Driver	DriverProfile, Vehicle, AvailabilitySlot, DriverLocation, DriverReview	PostgreSQL · schedules / driver data
6 · Surprise Basket	BasketListing, BasketDescription, BasketOrder, PickupCode, PaymentSimulation	MongoDB · flexible products and orders
7 · Common User + Notify	UserProfile, Notification, NotificationPreference, DeliveryAttempt, DeviceToken	PostgreSQL profiles; Keycloak identity

Service boundaries that must stay clear

• Offer Service alone changes stock and creates reservations. Matching and Marketplace use its API; never write into its database.

• Matching owns pickup/driver assignments. Driver Service owns driver profiles and availability.

• Partner Registry verifies organizations; Keycloak manages credentials and login.

• Impact Analytics consumes completed events; it must not directly read other services' tables.

Advanced requirement: DemandForecast (planned) provides forecasting; define a realistic rule-based baseline if ML is not ready.

03 Technical architecture

The component proposal aligns with the mandatory AWD microservice stack.

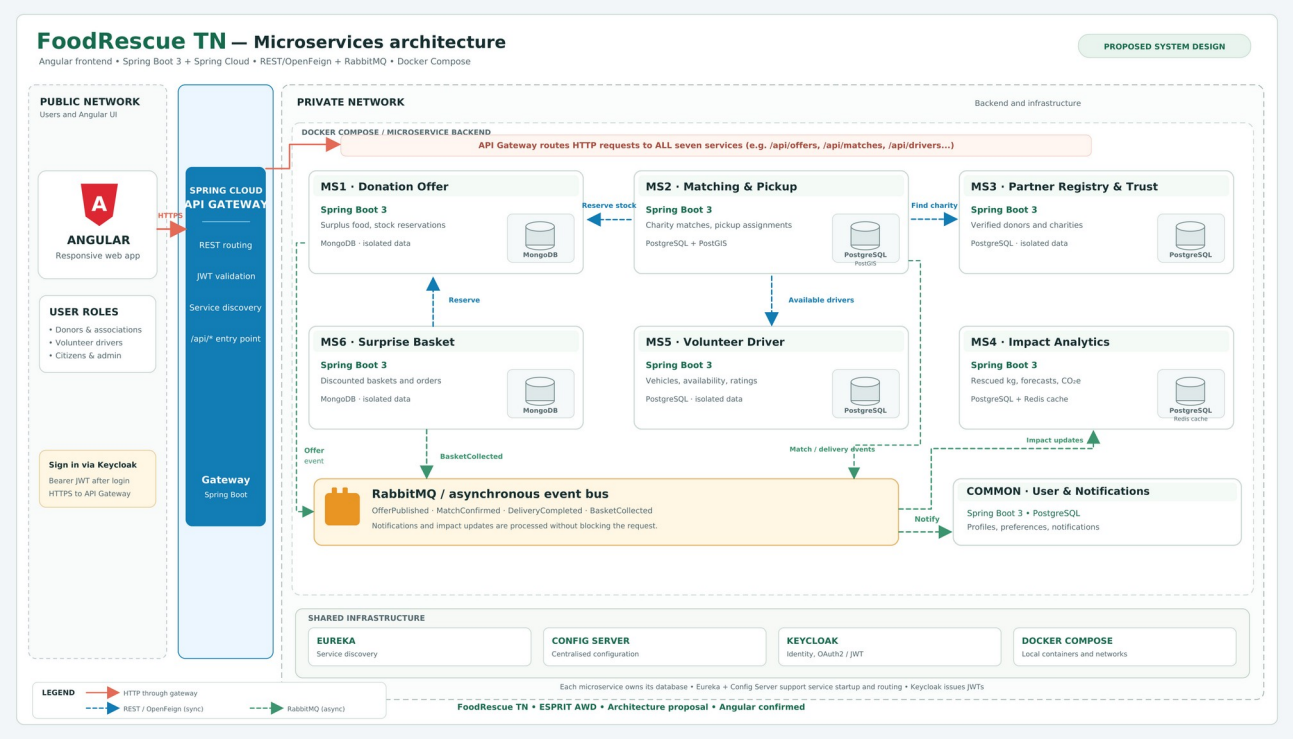


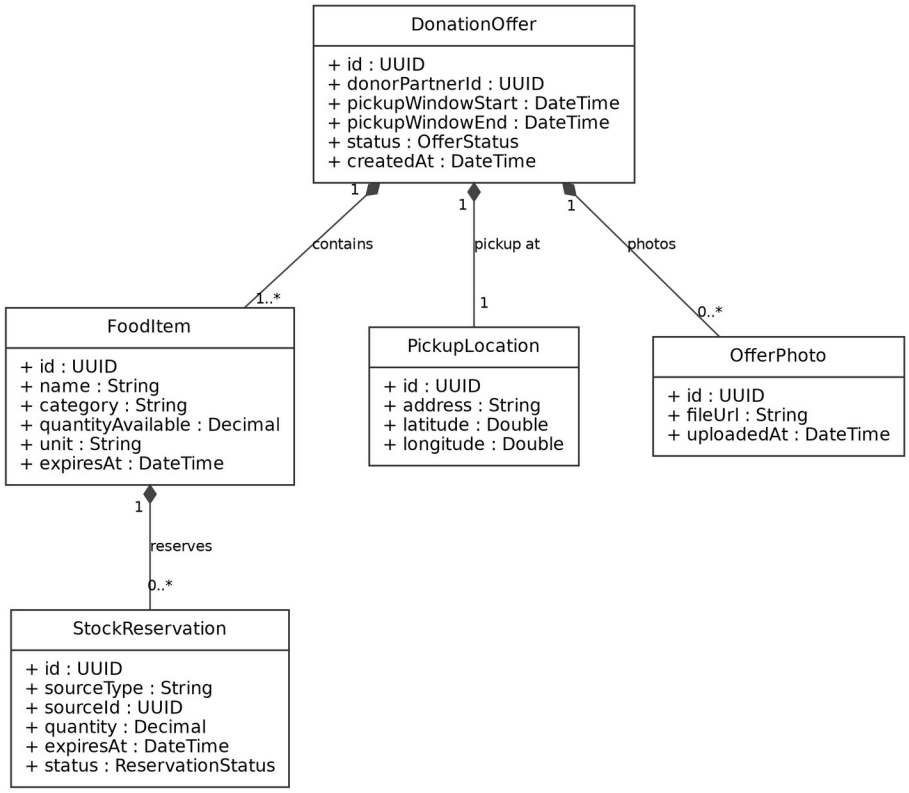
Figure 1 — Team-supplied Angular microservices architecture (original file preserved separately). The diagram includes Gateway, Eureka, Keycloak, Config Server, RabbitMQ, seven services, databases and Docker Compose.

Component	Choice / why
Frontend	Angular; planned browser interface for all roles
Backend	Spring Boot REST APIs; shared course ecosystem
Gateway / discovery	Spring Cloud Gateway / Eureka; central routing and service discovery
Configuration / security	Spring Config Server / Keycloak OAuth2 and JWT
Messaging	RabbitMQ; decoupled delivery and notification events
Resilience / deployment	Timeouts, retries and compensations; Docker Compose planned

04 UML class diagrams — A

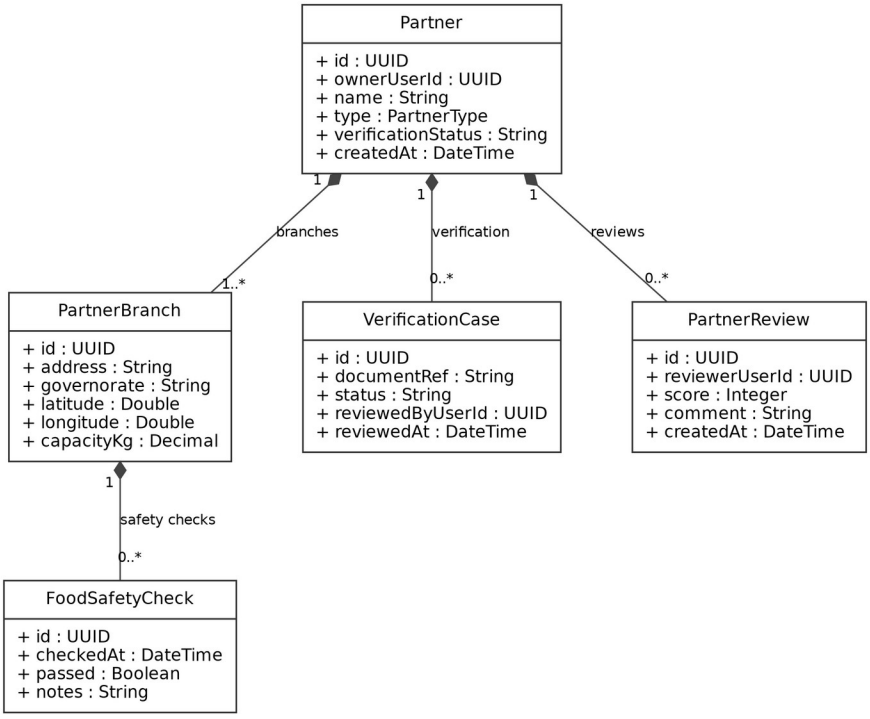
Five-class models kept within the owner microservice.

4.1 Donation Offer



Offer owns photos, pickup location, stock items and reservations.

4.2 Partner Registry & Trust

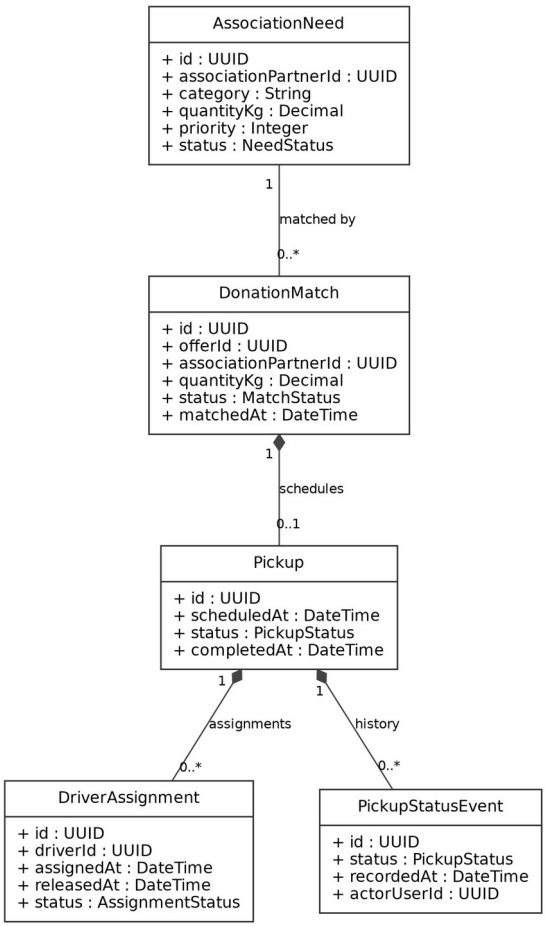


Partner owns branches, verification history, safety and reviews.

04 UML class diagrams — B

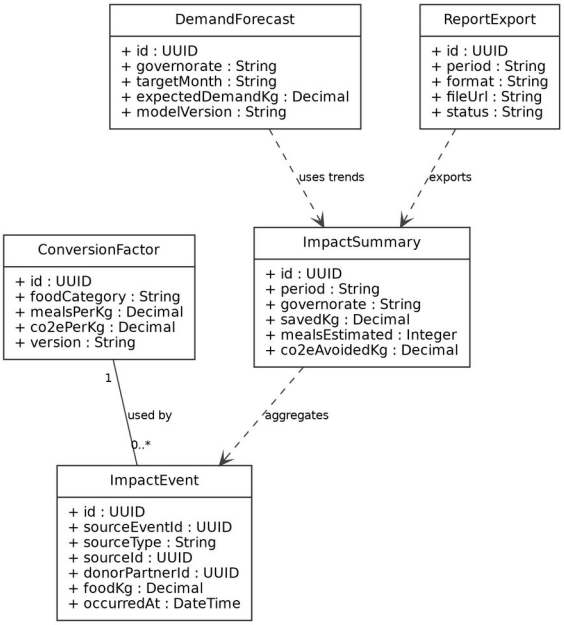
Classes in other microservices are referenced by IDs, not database foreign keys.

4.3 Matching & Pickup



Match, pickup, assignments, status history.

4.4 Impact Analytics



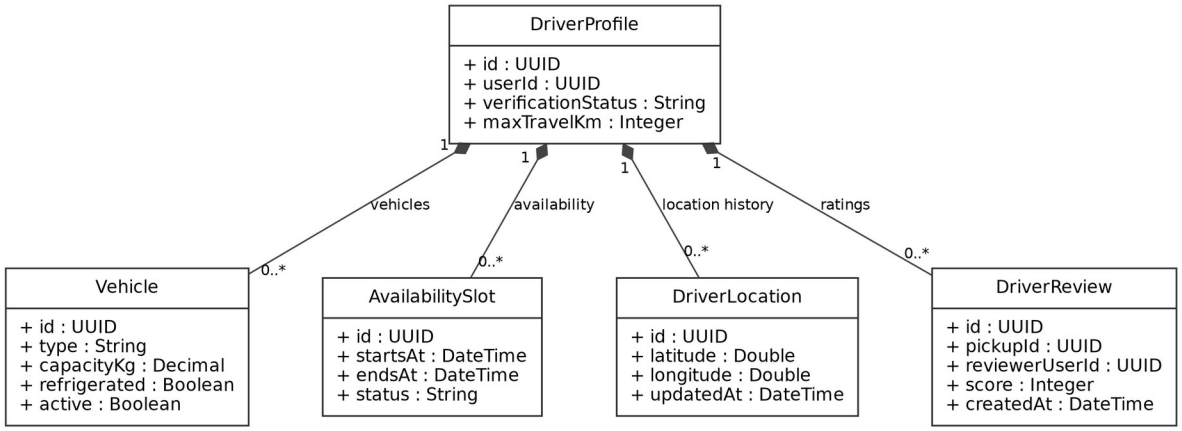
Event ledger, conversion factors, forecasts.

Design rule: only one ACTIVE DriverAssignment per pickup. ImpactEvent.sourceEventId prevents duplicate counting.

04 UML class diagrams — C

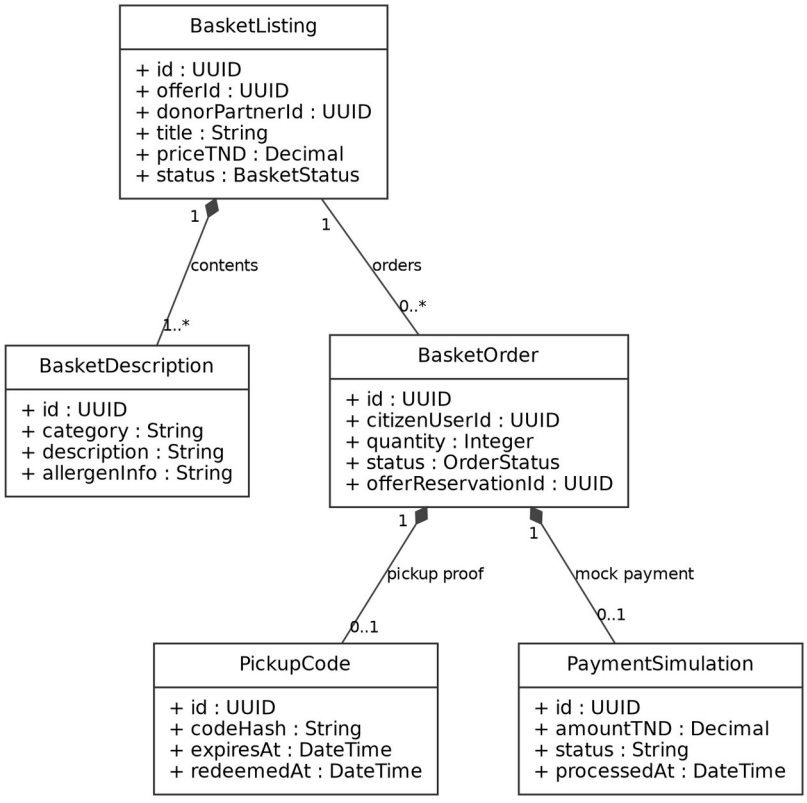
Driver availability and citizen marketplace models.

4.5 Volunteer Driver



Driver Service supplies candidate drivers but does not schedule pickups.

4.6 Surprise Basket Marketplace

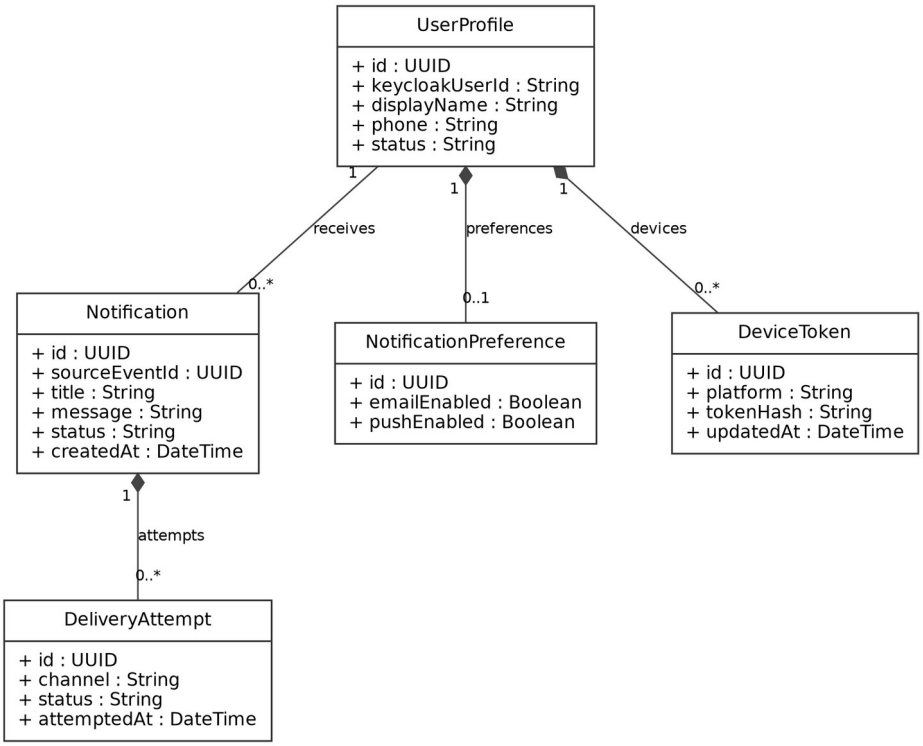


BasketOrder stores offerReservationId; stock stays in Donation Offer.

04 UML class diagrams — D

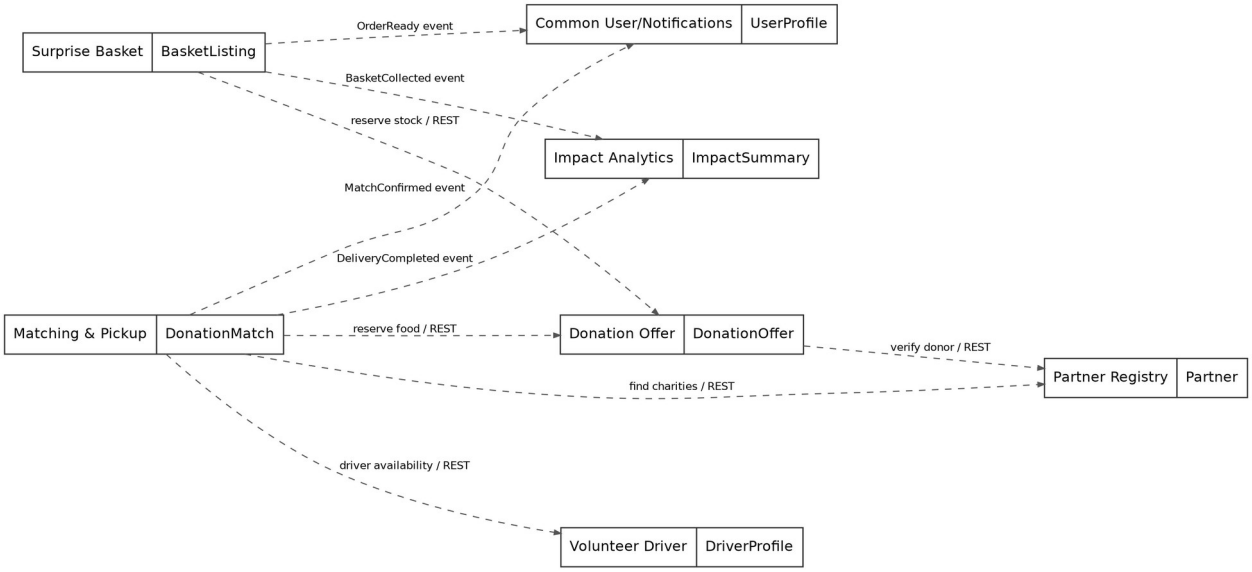
Common user service and a compact dependency overview.

4.7 Common User, Auth & Notification



Application profiles and notifications are modeled here; Keycloak owns credentials.

Service dependencies (not a class diagram)



Dashed arrows represent REST calls or RabbitMQ events between services, not UML class associations or shared database relationships. Each service has its own class diagram above.

05 Communication & sequence flows

Synchronous = a response is needed now. Asynchronous = publish an event and continue.

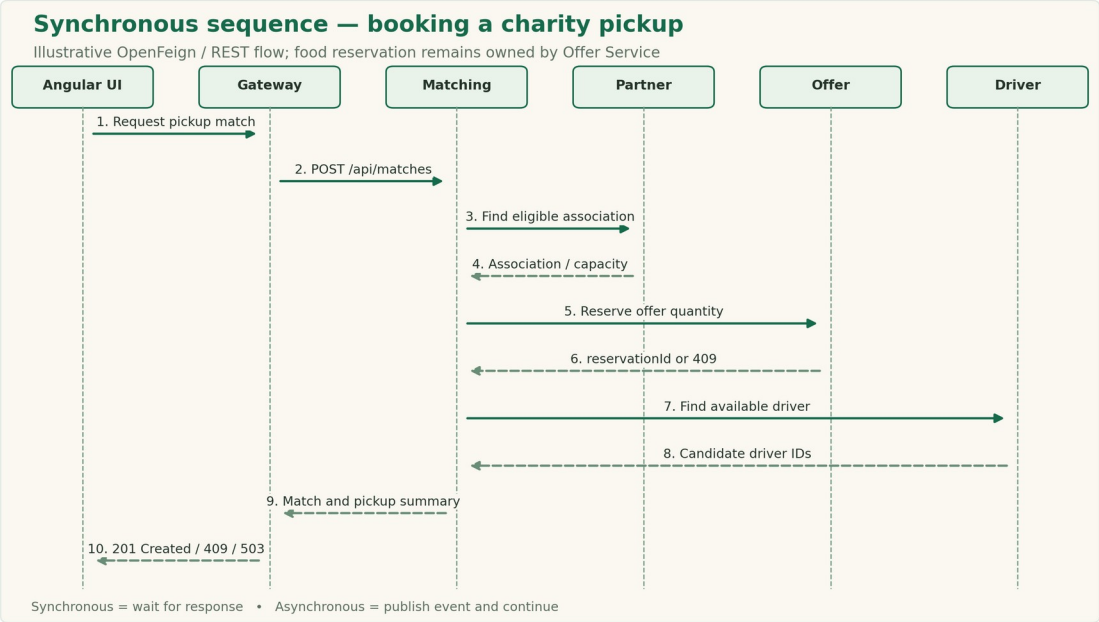


Figure 2 — REST / OpenFeign: Matching checks partner eligibility, reserves stock and requests available drivers.

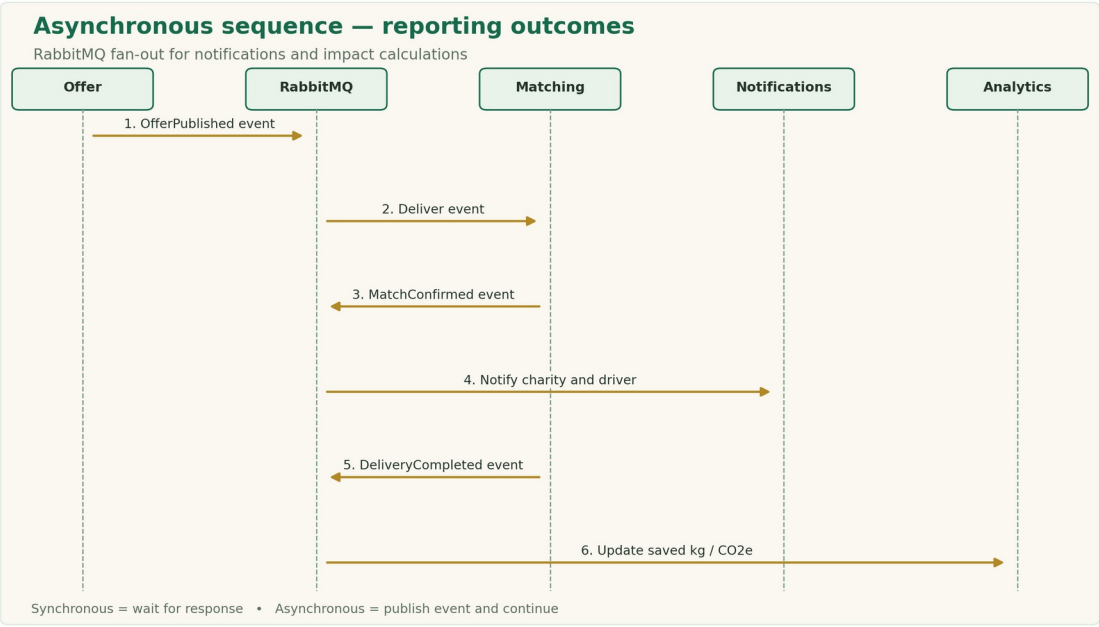


Figure 3 — RabbitMQ: delivery events trigger notifications and analytics independently.

Call / event	Data exchanged	Why / fallback
Matching → Offer (sync)	offerId, quantity, reservationId	Immediate reservation; 409 if unavailable
Matching → Driver (sync)	pickupId, available driverIds	Immediate candidates; pending if unavailable
DeliveryCompleted (async)	eventId, pickupId, foodKg	Analytics retries/deduplicates events

Failure handling: timeouts + limited retries; return 409 on stock conflict; release unused reservations after failed matching; deduplicate RabbitMQ events by event ID.

06 REST API contracts — A

Proposed endpoints; not yet implemented or Swagger-verified.

Donation Offer

Method	Endpoint	Purpose	HTTP
POST	/api/offers	Publish surplus offer	201, 400, 403
GET	/api/offers/{id}	Read offer and stock	200, 404
POST	/api/offers/{id}/reservations	Reserve food quantity	201, 400, 409
POST	/api/reservations/{id}/release	Release an unused reservation	200, 404, 409

Matching & Pickup

Method	Endpoint	Purpose	HTTP
POST	/api/matches	Match offer with charity	201, 404, 409
POST	/api/pickups/{id}/assign	Assign an available driver	200, 404, 409
PATCH	/api/pickups/{id}/status	Update delivery status	200, 403, 404
GET	/api/pickups?driverId=	List driver pickups	200, 401

Partner Registry & Trust

Method	Endpoint	Purpose	HTTP
POST	/api/partners	Register an organization	201, 400
GET	/api/partners/{id}	Read partner profile	200, 404
PATCH	/api/partners/{id}/verification	Approve or reject organization	200, 403, 404
POST	/api/partners/{id}/reviews	Submit a partner review	201, 400, 403

Impact Analytics

Method	Endpoint	Purpose	HTTP
GET	/api/impact/summary	Global saved-food totals	200
GET	/api/impact/governorates/{id}	Local impact summary	200, 404
GET	/api/impact/forecast?month=	Predicted demand (planned)	200, 400
GET	/api/impact/reports?period=	Export/report status	200, 400

06 REST API contracts — B

Short contracts for the remaining microservices and planned events.

Volunteer Driver

Method	Endpoint	Purpose	HTTP
POST	/api/drivers	Create volunteer profile	201, 400, 409
GET	/api/drivers/{id}	Read driver profile	200, 404
PUT	/api/drivers/{id}/availability	Update available time slots	200, 400, 403
GET	/api/drivers/available?lat=&lng=	Find available drivers	200, 400

Surprise Basket Marketplace

Method	Endpoint	Purpose	HTTP
GET	/api/baskets?governorate=	Browse discounted baskets	200
POST	/api/orders	Reserve and place order	201, 400, 409
GET	/api/orders/{id}	View order and pickup details	200, 403, 404
POST	/api/orders/{id}/collect	Redeem pickup code	200, 404, 409

Common User + Notification

Method	Endpoint	Purpose	HTTP
GET	/api/users/me	Read application profile	200, 401
PATCH	/api/users/me	Edit profile/preferences	200, 400, 401
GET	/api/notifications	List user notifications	200, 401
PATCH	/api/notifications/{id}/read	Mark notification read	200, 403, 404

Event contracts (RabbitMQ)

OfferPublished(offerId) • MatchConfirmed(matchId, partnerIds) • DeliveryCompleted(pickupId, foodKg) • BasketCollected(orderId, foodKg)